Especificación de Requisitos de Software

HuertoHogar · Versión 2.0 · Evaluación Parcial 2

Asignatura: DSY1104, Desarrollo Fullstack II. Equipo: Sebastian Monsalve y Lucas Parra. Fecha de esta versión: 20 de octubre de 2026.

1. Propósito y antecedentes

Actualizar la tienda HuertoHogar entregada en HTML, CSS y JavaScript a una aplicación frontend moderna con React. La propuesta conserva los siete productos, fotografías, video y enfoque en productos frescos del proyecto original. Incorpora componentes reutilizables, rutas, formularios controlados, estado compartido y pruebas automatizadas con Jasmine y Karma.

Este ERS describe la versión implementada. No se recibió un ERS anterior: este documento constituye la propuesta terminada de la versión frontend y no acredita requisitos adicionales de evaluaciones previas que no estén en los archivos recibidos.

2. Alcance

El visitante consulta productos, filtra el catálogo y utiliza el carrito. Puede registrar una cuenta de prueba e iniciar sesión. El administrador de demostración mantiene productos y roles de usuarios locales. El sistema presenta información de la tienda y guarda consultas de contacto en el navegador.

Quedan fuera de alcance el backend, base de datos compartida, pagos, despacho, recuperación de contraseña y correo real. La compra se confirma como simulación sin descontar stock ni crear un pedido persistente. La autorización de frontend es una ayuda visual del prototipo; no garantiza seguridad en producción.

3. Actores y entorno

Visitante: consulta contenido y usa carrito y formularios. Cliente: cuenta registrada con saludo y sesión local. Administrador: utiliza el panel de productos y usuarios. Desarrollador/evaluador: instala dependencias, ejecuta pruebas y revisa los reportes.

Entorno: navegador moderno con JavaScript y localStorage; Node.js compatible con Vite para desarrollo; npm con archivo de bloqueo. Internet requerido para instalar dependencias y navegador de pruebas; los recursos visuales y Bootstrap quedan incluidos en la compilación.

4. Requisitos funcionales y aceptación

ID	Requisito	Criterio de aceptación	Implementación
RF01	Navegar entre secciones	Las rutas cambian sin recarga; una ruta desconocida muestra página no encontrada	App, Layout, Content
RF02	Consultar catálogo	Se muestran siete productos iniciales con nombre, foto, precio y stock	Catalog, ProductCard
RF03	Filtrar y buscar	Categoría y nombre se combinan; sin coincidencias se informa resultado vacío	Catalog
RF04	Consultar detalle	El código en la ruta determina el producto; uno inexistente muestra aviso	Detail
RF05	Agregar al carrito	Acepta cantidades enteras positivas hasta stock; acumula productos repetidos	cart, Store
RF06	Mantener carrito	Cambia cantidades, elimina, vacía y calcula suma de precio por cantidad	Cart
RF07	Persistir estado	Carrito, catálogo y sesión se recuperan al recargar en el mismo navegador	storage, Store
RF08	Registrar cuenta	Valida campos, RUN módulo 11 y correo no duplicado antes de guardar	AccountForm, validation
RF09	Iniciar/cerrar sesión	Solo acepta credenciales registradas o administrador demo; salir limpia sesión	AccountForm, Layout
RF10	Contactar	Valida nombre, correo y comentario; confirma guardado local explícito	AccountForm
RF11	Mostrar información	Inicio con video, Nosotros, blog y dos artículos accesibles	Home, Content
RF12	Administrar productos	Admin crea/edita/elimina; evita código duplicado y precio/stock inválidos	Admin
RF13	Administrar usuarios	Admin consulta cuentas registradas, cambia rol o elimina	Admin
RF14	Confirmar simulación	Solo con carrito no vacío; limpia carrito e indica que no hay cobro ni envío	Cart

5. Reglas de negocio

Productos: código alfanumérico en mayúsculas de 3 a 12 caracteres, nombre y descripción obligatorios, precio mayor que cero, stock entero no negativo. La interfaz de administración selecciona una de las imágenes locales. El carrito rechaza cantidades que superen stock; al editar un producto sincroniza su precio y stock con el carrito, limita cantidades o retira el artículo si queda sin stock. Eliminar un producto también lo retira del carrito.

Cuenta: nombre válido de 2 a 100 caracteres; correo hasta 100 caracteres de dominios duoc.cl, profesor.duoc.cl o gmail.com; contraseña de prueba entre 4 y 10 caracteres, conforme a las reglas de la base recibida; RUN validado por módulo 11, con o sin puntos y guion; dirección obligatoria hasta 200 caracteres. Los correos se normalizan a minúsculas. Las cuentas nuevas tienen rol cliente. Región y comuna deben pertenecer al listado del prototipo, que incluye Metropolitana, Valparaíso y Biobío con comunas de muestra.

Contacto: nombre, correo permitido y mensaje no vacío de hasta 500 caracteres. Se registra solo localmente y se informa que no se envió un correo. Las contraseñas de prueba se almacenan localmente para la simulación de login; el objeto de sesión solo contiene nombre, correo y rol.

6. Requisitos no funcionales

ID	Requisito	Evidencia o criterio
RNF01	React modular	Componentes funcionales, props, useState, useEffect y Context
RNF02	Bootstrap responsivo	Grid col-12/col-sm-6/col-lg-4, menú adaptable y table-responsive
RNF03	Accesibilidad básica	Etiquetas de campos, estados aria, avisos role y navegación con enlaces/botones
RNF04	Pruebas reproducibles	npm ci, npm test y npm run build con versiones bloqueadas
RNF05	Cobertura mínima	Sentencias y líneas >=90%; ramas y funciones >=85%; Karma falla si no se alcanza
RNF06	Manejo de almacenamiento	Lectura corrupta retorna alternativa; escritura fallida informa resultado cuando el formulario guarda cuenta o mensaje
RNF07	Hosting estático	HashRouter y base relativa; no requiere reescritura de rutas del servidor

7. Arquitectura y flujo de datos

App configura rutas dentro de StoreProvider. Layout reúne navegación, contador, avisos y pie. Catalog filtra por estado local y pasa cada producto y callback a ProductCard. ProductCard no conoce el almacenamiento: comunica intención de agregar con onAdd. Store centraliza items, products y user; usa funciones puras para validar operaciones y useEffect para persistir cambios. AccountForm selecciona campos y validaciones según la propiedad mode. Los servicios aíslan reglas que pueden probarse sin renderizar una página.

Se usan actualizaciones inmutables con map, filter y spread para que React detecte cambios. El detalle consulta useParams. Los campos de formulario se controlan con value y onChange, y se muestran errores junto a los datos correspondientes.

8. Restricciones y riesgos conocidos

El estado pertenece a un navegador y origen; no se comparte entre equipos ni se sincroniza entre pestañas. La eliminación de un usuario o cambio de rol se refleja al volver a iniciar sesión; no revoca otras sesiones abiertas. El carrito usa copias de productos hasta que se modifican desde el panel en la misma sesión. El pedido simulado no reduce existencias. El rol local se puede modificar desde herramientas de desarrollo, por lo que este prototipo no debe usarse como autenticación de producción.

No se garantiza persistencia si el navegador bloquea almacenamiento o se queda sin espacio. Las pruebas cubren recuperación de JSON corrupto y errores de escritura. Para producción se requeriría autenticación de servidor, almacenamiento seguro y validación de las operaciones en backend.

9. Verificación y entrega

Se ejecutaron 46 casos automatizados y la compilación de producción. Se revisaron ocho rutas en anchos de 390, 768 y 1440 píxeles, sin desbordes finales ni errores de JavaScript observados. El informe de cobertura contiene resultados numéricos, matriz de casos, exclusiones y limitaciones.

La entrega incluye código fuente, package-lock.json, configuración Karma, pruebas, recursos y los documentos ERS y cobertura. La compilación y los reportes de cobertura se generan durante la ejecución y no se incluyen como carpetas duplicadas. El enlace público de GitHub debe agregarse al entregar después de subir el contenido al repositorio del equipo. El ZIP por sí solo no satisface ese enlace.